

Problem 1

For problem 5.(d)

Since $Z(n) \in \Theta(n)$, there exist constants $c_1, c_2 > 0$ and $n_0 > 0$ such that:

$$c_1 n \leq Z(n) \leq c_2 n, \quad \text{for all } n \geq n_0.$$

Since $Y(n) \in \Theta(n \log n)$, there exist constants $c_3, c_4 > 0$ and $n_1 > 0$ such that:

$$c_3 n \log n \leq Y(n) \leq c_4 n \log n, \quad \text{for all } n \geq n_1.$$

Using the bounds for $Z(n)$ and $Y(n)$, we find:

$$\frac{c_1 n}{c_4 n \log n} \leq \frac{Z(n)}{Y(n)} \leq \frac{c_2 n}{c_3 n \log n}, \quad \text{for all } n > \max(n_0, n_1)$$

Simplifying the bounds:

$$\frac{c_1}{c_4 \log n} \leq \frac{Z(n)}{Y(n)} \leq \frac{c_2}{c_3 \log n}, \quad \text{for all } n > \max(n_0, n_1)$$

As $n \rightarrow \infty$, $\log n \rightarrow \infty$, so both the lower and upper bounds approach 0:

$$\frac{c_1}{c_4 \log n} \rightarrow 0 \quad \text{and} \quad \frac{c_2}{c_3 \log n} \rightarrow 0.$$

By the Squeeze Theorem:

$$\lim_{n \rightarrow \infty} \frac{Z(n)}{Y(n)} = 0.$$

Since $\lim_{n \rightarrow \infty} \frac{Z(n)}{Y(n)} = 0$, we conclude that $Z(n) \in O(Y(n))$. Thus, for all large enough n , there is an input of size n for which Z runs faster than Y .

This is my solution and instead of saying $Z(n) \in O(Y(n))$ this does guarantee strictly larger. Instead, I should have been more clear because big-O only guarantee's less than or equal to not always above. I should have argued that since the limit is zero and both functions are strictly positive at some point Y will always be bigger than Z this is a heuristic argument since I do not want to get into analysis on this problem.

Problem 2

- (a) The potential function does not work since for example if I started with 10 of each chip so 20 in total in the pile and I first took a maize chip. Then I would add 3 blue chips back this gives us a total pile size of 22.

Instead, use the potential function: $n_i = 4m_i + b_i$ s.t. m_i is the number of maize chips at round i and b_i is the number of blue chips at round i . Where the time unit is a round.

- If we take a maize chip and in the worst case add 3 blue chips back:

$$n_{i+1} = 4(m_i - 1) + b_i + 3 = 4m_i + b_i - 1 < n_i = 4m_i + b_i$$

- If we take a blue chip:

$$n_{i+1} = 4m_i + b_i - 1 < n_i = 4m_i + b_i$$

Also, we are lower bounded by zero since the min chips of each color we can have is zero thus $n_i \geq 0$ for all $i \in \mathbb{N}$

Thus, this game halts.

- (b) This potential function does not work since we would have that $d_i = \frac{d_{i-1} + 1}{2} = \frac{d + 2^n - 1}{2^n}$.

Fix $c > 0$ as our fixed decreasing difference.

By the Archimedean property fix an $N \in \mathbb{N}$ s.t.

$$\frac{1}{N+1} < \frac{1}{\log_2 \frac{d-3}{c}}$$

Thus we get that:

$$\frac{d-3}{2^{N+1}} < c$$

Now consider $d_n - d_{n+1}$ s.t. $n \geq N+1$

$$\begin{aligned} d_n - d_{n+1} &= \frac{d + 2^n - 1}{2^n} - \frac{d + 2^{n+1} - 1}{2^{n+1}} \\ &= \frac{d-3}{2^{n+1}} < \frac{d-3}{2^{N+1}} < c \end{aligned}$$

Thus we have shown that it does not decrease by a fixed amount for any $c > 0$

Now to show that this game will never stop. So using the same function as above we can take

the limit and see if the distance every gets to zero.

$$\lim_{n \rightarrow \infty} \frac{d + 2^n - 1}{2^n} = \lim_{n \rightarrow \infty} \frac{d}{2^n} + 1 - \frac{1}{2^n} = 1$$

Thus the distance never actually becomes zero, and we never reach the target.

Problem 3

(a) Consider the input $a = 6, b = 2$.

On the first pass we get that $6 > 2$ thus a becomes 3 and b becomes 4

On the second pass we get that $4 > 3$ thus a becomes 6 and b becomes 2

Thus, we are at our initial inputs again and this is a cycle thus we never stop.

(b) Define our time unit as calling *ALG* one time. Let our potential function be $a - b$.

- First we can assume that $a > b$ since if it were not then *ALG* would exit immediately
- (Decrease by constant amount).

– If b is even:

$$a - 1 - b - 1 = a - b - 2 < a - b$$

– If b is odd:

$$a - b - 1 = a - b - 1 < a - b$$

Thus we decrease by at least one for each case.

- (Lower bound) The lower bound is 0 since if $a - b < 0 \implies a < b \implies a \leq b$ thus our function would stop so $a - b \geq 0$

Thus our function would halt.

(c) Define our time unit as one loop of the for loop. Let our potential function be:

$$f(x) = \begin{cases} x & \text{if } x \text{ is even.} \\ x + 4 & \text{if } x \text{ is odd.} \end{cases}$$

- First we can assume that $x > 10$ since if not *Foo* would end.
- (Decrease by constant amount).
 - If x_{i-1} is odd meaning x on the $i - 1$ th loop. This implies that x_i is even, since $x_i = x_{i-1} + 3$

$$f(x_{i-1}) - f(x_i) = x_{i-1} + 4 - x_i = x_{i-1} + 4 - x_{i-1} - 3 = 1$$

– If x_{i-1} is even

* If x_i is even

$$f(x_{i-1}) - f(x_i) = x_{i-1} - x_i = x_{i-1} - x_{i-1}/2 = x_{i-1}/2 > 10/2 > 1$$

* If x_i is odd

$$f(x_{i-1}) - f(x_i) = x_{i-1} - x_i - 4 = x_{i-1} - x_{i-1}/2 - 4 = x_{i-1}/2 - 4 > 10/2 - 4 = 1$$

Thus we decrease by at least one each time since each difference is greater than one.

- (Lower bound) The lower bound is 6. Since if $f(x) < 6$ this implies that $x < 10$ since either $f(x) = x < 6$ or $f(x) = x + 4 < 10 \implies x < 6 < 10$. Thus, if our function is less than 6 then we would have stopped.

Problem 4

(a) The recurrence relation for this program is:

$$2T(n/2) + T(n-1) + O(1)$$

The reason is we do 2 calls to slow sort on half the array in lines 2,3. We then compare and swap elements which is $O(1)$. Then on line 6 we call slow sort on $n-1$ elements in the array which gives us $T(n-1)$.

This cannot be analyzed with the master theorem because of the term $T(n-1)$, which is not of the form $kT(n/b) + O(n^d \log^w n)$

(b) The recurrence relation for this problem is:

$$3T(2n/3) + O(1)$$

The reason is we do $O(1)$ work in lines 2 – 5. We then call the function on elements 1 to $t = 2n/3$ which is $2n/3$ elements. We then call it on elements $n-t+1$ to n which is $2n/3$ elements then again on 1 to t thus we call $3T(2n/3)$

We can analyze using the master theorem. Use $k = 3, b = 3/2, d = 0, w = 0$. We get that $T(n) \in O(n^{\log_{3/2}(3)})$

Problem 5

(a)

$$\begin{aligned}
 A(x) \cdot B(x) &= (p(x)x^{n/2} + q(x))(r(x)x^{n/2} + s(x)) \\
 &= p(x)r(x)x^n + p(x)s(x)x^{n/2} + q(x)r(x)x^{n/2} + q(x)s(x) \\
 &= p(x)r(x)x^n + (p(x)s(x) + q(x)r(x))x^{n/2} + q(x)s(x)
 \end{aligned}$$

(b)

$$(p(x) + q(x))(s(x) + r(x)) - p(x)r(x) - q(x)s(x) = (p(x)s(x) + q(x)r(x))$$

$$\begin{aligned}
 A(x) \cdot B(x) &= p(x)r(x)x^n + (p(x)s(x) + q(x)r(x))x^{n/2} + q(x)s(x) \\
 &= p(x)r(x)x^n + ((p(x) + q(x))(s(x) + r(x)) - p(x)r(x) - q(x)s(x))x^{n/2} + q(x)s(x) \\
 &= p(x)r(x)(x^n - x^{n/2}) + (p(x) + q(x))(s(x) + r(x))x^{n/2} + q(x)s(x)(1 - x^{n/2})
 \end{aligned}$$

(c) Base Case: If the degree is 0 then $A(x)B(x) = a_0b_0$

Then split $A(x), B(x)$ the same way with $A(x) = p(x)x^{n/2} + q(x)$, $B(x) = r(x)x^{n/2} + s(x)$. This implies that each polynomial is less than or equal to degree $n/2 - 1$

Then compute the following products:

$$U = p(x)r(x) \quad V = q(x)s(x) \quad W = (p(x) + q(x))(s(x) + r(x))$$

Then using the proof that $A(x) \times B(x) = Ux^n + (W - U - V)x^{n/2} + V$ we only have to do three products then return that result.

- (Proof of correctness):

Proof. Base Case: If the degree is 0 then you just multiply constants so $AB = a_0b_0$

Inductive Step: Assume that we can multiply for all degrees less than $n \in \mathbb{N}$

Consider AB : Using (b) we can show we can break it down into that form with $p(x), r(x), q(x), s(x)$.

We then can compute those multiplications of $pr, (p + q)(s + r), qs$ by the inductive step thus we just plug them back into the formula, and we get our product.

Thus, by induction we can multiply for all $n \in \mathbb{N}$ □

- (Runtime) In our algorithm we can ignore the base case since we do $O(1)$ work. Then we do 3 calls to the product function on polynomials of degree less than $n/2$ thus we get $3T(n/2)$. It then takes $O(n)$ to sum up the polynomials since we do term by term, and

they could be n terms.

Thus, we get:

$$T(n) = 3T(n/2) + O(n)$$

Thus, by the master theorem we get that $T(n) \in O(n^{\log_2(3)})$

Problem 5

- (a) A simple proof force algorithm is to first start at index 1. Then count all the numbers smaller than it after its index. We then do this for all indices. Thus, we get about $1 + 2 + 3 + \dots + n \approx n^2$ thus we get $\Theta(n^2)$

(b) 1: **function** MERGEANDCOUNT(A)
 2: $n \leftarrow$ Size of A
 3: $merge \leftarrow []$
 4: $e \leftarrow 0$
 5: $first \leftarrow 1$
 6: $second \leftarrow$ index of the first element in the second half
 7: **while** $first <$ index of the first element in the second half & $second < n + 1$ **do**
 8: **if** $A[first] < A[second]$ **then**
 9: $merge$ add $A[first]$ to the end
 10: $first \leftarrow first + 1$
 11: **else if** $A[second] < A[first]$ **then**
 12: $merge$ add $A[second]$ to the end
 13: $second \leftarrow second +$ index of last element in first half $- first$
 14: $e \leftarrow e + 1$
 15: **end if**
 16: **end while**
 17: $A = merge$
 18: **return** e
 19: **end function**

- (Proof of correctness) Case 1: ($A[first] < A[second]$)

Since the two halves of the array are sorted, we know that $A[first]$ is the largest value we have looked at so far in the first half. This means:

$$A[1] < A[2] < \dots < A[first] < A[second].$$

Thus, for all elements in the first half up to $A[first]$, there are no errors caused by $A[second]$, as we are maintaining the "least to greatest" order. Consequently, we can safely move the pointer $first$ forward without adding any errors.

- Case 2: ($A[first] > A[second]$):

Since the two halves of the array are sorted, we know:

$$A[second] < A[first] < A[first+1] < \dots$$

This implies that $A[second]$ will cause errors for all remaining elements in the first half

starting from $A[\text{first}]$. Therefore, we add:

index of first element in the second half – first

to the total error count. After accounting for these errors, we move the pointer second forward.

- Termination:

- Case 1: The first pointer reaches the end of the first half.

At this point, all elements in the first half have been processed. Since all remaining elements in the second half are greater than any already processed elements in the first half (due to the sorted order), no further errors will occur.

- Case 2: The second pointer reaches the end of the second half.

At this point, all elements in the second half have been processed. Since all remaining elements in the first half are greater than any already processed elements in the second half (due to the sorted order), all remaining elements in the first half will not cause additional errors since there is no elements left to cause errors.

Thus, we have proved this algorithm is complete.

Since we move either *first* or *second* each time by one, and we only look at each element a single time and never go backwards. We can see that we can do at most n loops. Then in all the other places except line 16 we do $O(1)$ work, but in line 16 we do $O(n)$ thus the algorithm is $O(n)$.

(c) 1: **function** COUNTERRORSRECURSIVE(A)
 2: **if** Size of $A \leq 1$ **then**
 3: **return** 0 ▷ Base case: no errors in a single element
 4: **end if**
 5: $mid \leftarrow \lfloor \text{Size of } A / 2 \rfloor$
 6: $first \leftarrow A[0, 1, 2, \dots, mid]$
 7: $second \leftarrow A[mid, mid + 1, \dots, n]$
 8: $first_errors \leftarrow \text{CountErrorsRecursive}(first)$
 9: $second_errors \leftarrow \text{CountErrorsRecursive}(second)$
 10: $A \leftarrow first + second$ ▷ Merge the two arrays so first is on the left and second is on the right
 11: $cross_errors \leftarrow \text{MergeAndCount}(A)$
 12: **return** $first_errors + second_errors + cross_errors$
 13: **end function**

- (Proof of Correctness) Base Case: There is only one element so only one way to put the hard drives in, so there are no errors.

- Inductive Hypothesis: Assume that the algorithm correctly counts errors for all arrays of size $n \leq k$, where $k \geq 1$.

The input array arr of size $n = k + 1$ is split into two halves:

Left half: $A[0, 1, \dots, mid]$ (size $\approx n/2$)

Right half: $A[min, mid + 1, \dots, n]$ (size $\approx n/2$).

By the inductive hypothesis, the recursive calls:

CountErrorsRecursive(first) and CountErrorsRecursive(second)

correctly count all errors within the left and right halves, respectively.

- Merging and Counting Cross Errors: The **MergeAndCount** function merges the two sorted halves and counts cross errors, which involve one element from the left half and one element from the right half.

During the merge step:

- If $A[first] < A[second]$, $A[first]$ is appended to the merged array.
- If $A[first] > A[second]$, $A[second]$ is appended to the merged array.

Then we return A to be a sorted array, since we add the least element each time and do not repeat. The we can assume that the arrays *first* and *second* are sorted, then we can combine them so the first and second halves are sorted.

The total number of errors in arr is:

$$\text{total_errors} = \text{left_errors} + \text{right_errors} + \text{cross_errors}.$$

The reason why we do not over count is we merge and sort the arrays, By sorting we eliminate all the errors and now only have to count the errors between the two sub arrays that will be there regardless of sorting when we combine the two halves. Thus, by induction, the algorithm correctly counts errors for any array of size n for all $n \in \mathbb{N}$.

To show that the algorithm is $O(n \log n)$, we can use the Master Theorem.

The base case takes $O(1)$. Lines 5–7 take $O(n)$ to copy the arrays. Lines 8–9 result in $2T(n/2)$, as the function is recursively called twice on subarrays of size $n/2$. Line 10 takes $O(n)$ by the previous problem, Line 11 also takes $O(n)$. Adding these together takes $O(1)$.

Thus, the recurrence relation is:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n).$$

Using the Master Theorem:

$$T(n) \in O(n \log n).$$

Problem 6

(a) Define the time unit as one loop of the while loop. Define our potential function as remaining elements of V that have not been in S . Thus, initially our function is $|V|$.

- The lower bound is zero because we cannot process more than $|V|$ unique elements.
- A vertex t is added to S in line 13 only if it is in neither A nor B . Once t is added to S , it is placed in either A or B in line 14. Therefore, t cannot satisfy the condition in line 13 again, as it will belong to A or B . Additionally, vertices are removed from S after processing in line 8, and no mechanism in the algorithm allows a vertex to be removed then re-added to S . Thus, by construction, each vertex is added to S at most once, as required.

Thus, this potential function will always decrease by 1 since we will process each element once with no repeats, or we will exit early.

Thus, our potential function shows that we will eventually halt.

(b) This function determines if a graph is bipartite or not.